

Sets

```
In [1]: myset = {1, 2, 3, 4, 5}    # Set of numbers
myset
```

```
Out[1]: {1, 2, 3, 4, 5}
```

```
In [2]: len(myset)    # Length of the set
```

```
Out[2]: 5
```

```
In [3]: my_set = {1, 1, 2, 2, 3, 4, 5, 5}    # Duplicate elements are not allowed
my_set
```

```
Out[3]: {1, 2, 3, 4, 5}
```

```
In [4]: myset1 = {1.79, 2.08, 3.99, 4.56, 5.45}    # Set of float numbers
myset1
```

```
Out[4]: {1.79, 2.08, 3.99, 4.56, 5.45}
```

```
In [5]: myset2 = {'Asif', 'John', 'Tyrion'}    # Set of strings
myset2
```

```
Out[5]: {'Asif', 'John', 'Tyrion'}
```

```
In [6]: myset3 = {10, 20, "Hola", (11,22,33)}    # Mixed datatypes
myset3
```

```
Out[6]: {(11, 22, 33), 10, 20, 'Hola'}
```

```
In [7]: myset3 = {10, 20, "Hola", [11,22,33]}    # Set doesn't allow multiple items like
myset3
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 myset3 = {10, 20, "Hola", [11,22,33]}    #Set doesn't allow multiple items
like list
      2 myset3

TypeError: cannot use 'list' as a set element (unhashable type: 'list')
```

```
In [ ]: myset4 = set()    # Create an empty set
print(type(myset4))
```

```
In [ ]: my_set1 = set(('one', 'two', 'three', 'four'))
my_set1
```

Loop through a Set

```
In [8]: myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
```

```
for i in myset:  
    print(i)
```

four
five
three
two
eight
six
one
seven

```
In [9]: for i in enumerate(myset):  
        print(i)
```

(0, 'four')
(1, 'five')
(2, 'three')
(3, 'two')
(4, 'eight')
(5, 'six')
(6, 'one')
(7, 'seven')

Set Membership

```
In [10]: myset
```

```
Out[10]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [11]: 'one' in myset    # Check if 'one' exist in the set
```

```
Out[11]: True
```

```
In [12]: 'ten' in myset    # Check if 'ten; exist in the set
```

```
Out[12]: False
```

```
In [13]: if 'three' in myset:    # Check if 'three' exist in the set  
        print('Three is present in the set')  
else:  
    print('Three is not present in the set')
```

Three is present in the set

```
In [14]: if 'eleven' in myset:    # Check if 'eleven' exist in the set  
        print('Eleven is present in the set')  
else:  
    print('Eleven is not present in the set')
```

Eleven is not present in the set

Add & Remove Items

```
In [15]: myset
```

```
Out[15]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [16]: myset.add('NINE')    # Add item to a set using add() method
myset
```

```
Out[16]: {'NINE', 'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [17]: myset.update(['TEN', 'ELEVEN', 'TWELVE'])    # Add multiple items to a set using
myset
```

```
Out[17]: {'ELEVEN',
          'NINE',
          'TEN',
          'TWELVE',
          'eight',
          'five',
          'four',
          'one',
          'seven',
          'six',
          'three',
          'two'}
```

```
In [18]: myset.remove('NINE')    # Remove item in a set using remove() method
myset
```

```
Out[18]: {'ELEVEN',
          'TEN',
          'TWELVE',
          'eight',
          'five',
          'four',
          'one',
          'seven',
          'six',
          'three',
          'two'}
```

```
In [19]: myset.discard('TEN')    # Remove item from a set using discard() method
myset
```

```
Out[19]: {'ELEVEN',
          'TWELVE',
          'eight',
          'five',
          'four',
          'one',
          'seven',
          'six',
          'three',
          'two'}
```

```
In [20]: myset.discard('THIRTEEN')    # Don't show error even if the item isn't available
myset
```

```
Out[20]: {'ELEVEN',
          'TWELVE',
          'eight',
          'five',
          'four',
          'one',
          'seven',
          'six',
          'three',
          'two'}
```

```
In [21]: myset.clear()    # Delete all items in a set
         myset
```

```
Out[21]: set()
```

```
In [22]: del myset      # Delete the set object
         myset
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[22], line 2
      1 del myset #Delete the set object
----> 2 myset

NameError: name 'myset' is not defined
```

Copy Set

```
In [23]: myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
         myset
```

```
Out[23]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [24]: myset1 = myset    # Create a new reference "myset1"
         myset1
```

```
Out[24]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [25]: id(myset), id(myset1)    # The addresses of both myset & myset1 will be the same
```

```
Out[25]: (2285641284992, 2285641284992)
```

```
In [26]: my_set = myset.copy()    # Create a copy of the List
         my_set
```

```
Out[26]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [27]: id(my_set)    # The address of my_set will be different from myset because my_set
```

```
Out[27]: 2285641285216
```

```
In [28]: myset.add('ninne')
         myset
```

```
Out[28]: {'eight', 'five', 'four', 'ninne', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [29]: myset1    # myset1 will be also impacted as it is pointing to the same Set
```

```
Out[29]: {'eight', 'five', 'four', 'ninne', 'one', 'seven', 'six', 'three', 'two'}
```

```
In [30]: my_set    # Copy of the set won't be impacted due to changes made on the orginal
```

```
Out[30]: {'eight', 'five', 'four', 'one', 'seven', 'six', 'three', 'two'}
```

Set Operation

(i) Union

```
In [31]: A = {1, 2, 3, 4, 5}
        B = {4, 5, 6, 7, 8}
        C = {8, 9, 19}
```

```
In [32]: A | B    # Union of A and B (ALL elements from both sets, NO DUPLICATES)
```

```
Out[32]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [33]: A.union(B)    # Union of A and B
```

```
Out[33]: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
In [34]: A.union(B, C)    # Union of A, B and C
```

```
Out[34]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 19}
```

```
In [35]: """
Updates the set calling the update() method with union of A, B & C.

For below example Set A will be updated with union of A, B & C.
"""
A.update(B,C)
A
```

```
Out[35]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 19}
```

(ii) Intersection

```
In [36]: A = {1, 2, 3, 4, 5}
        B = {4, 5, 6, 7, 8}
```

```
In [37]: A & B    # Interaction of A and B (Common items in both sets)
```

```
Out[37]: {4, 5}
```

```
In [38]: A.intersection(B)
```

```
Out[38]: {4, 5}
```

```
In [39]: Intersection of A and B
```

```
Cell In[39], line 1
      Intersection of A and B
      ^
SyntaxError: invalid syntax
```

```
In [40]: """
Updates the set calling the intersection_update() method with the intersection o

For below example Set A will be upload with the intersection of A & B.
"""
A.intersection_update(B)
A
```

```
Out[40]: {4, 5}
```

(iii) Difference

```
In [41]: A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
```

```
In [42]: A - B    # Set of elements that are only in A but not in B
```

```
Out[42]: {1, 2, 3}
```

```
In [43]: A.difference(B)    # Difference of sets
```

```
Out[43]: {1, 2, 3}
```

```
In [44]: B - A    # Set of elements that are only in B but not in A
```

```
Out[44]: {6, 7, 8}
```

```
In [45]: B.difference(A)
```

```
Out[45]: {6, 7, 8}
```

```
In [46]: """
Updates the set calling the difference_update() method with the difference of se

For below example Set B will be updated with the difference of B & A.
"""
B.difference_update(A)
B
```

```
Out[46]: {6, 7, 8}
```

(iv) Symmetric Difference

```
In [47]: A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
```

```
In [48]: A ^ B    # Symmetric difference (Set of elements in A and B but not in both.
```

```
Out[48]: {1, 2, 3, 6, 7, 8}
```

```
In [49]: A.symmetric_difference(B)    # Symmetric difference of sets
```

```
Out[49]: {1, 2, 3, 6, 7, 8}
```

```
In [50]: """
Updates the set calling the symmetric_difference_update() method with the symmet

For below example Set A will be updated with the symmetric difference of A & B.
"""
A.symmetric_difference_update(B)
```

Subset, Superset & Disjoint

```
In [51]: A = {1, 2, 3, 4, 5, 6, 7, 8, 9}
B = {3, 4, 5, 6, 7, 8}
C = {10, 20, 30, 40}
```

```
In [52]: B.issubset(A)    # Set B is said to be the subset of set A if all elements of B
```

```
Out[52]: True
```

```
In [53]: A.issuperset(B)    # Set A is said to be the superset of set B if all elements o
```

```
Out[53]: True
```

```
In [54]: C.isdisjoint(A)    # Two sets are said to be disjoint sets if they have no commo
```

```
Out[54]: True
```

```
In [55]: B.isdisjoint(A)    # Two sets are said to be disjoint sets if they have no commo
```

```
Out[55]: False
```

Other Builtin Functions

```
In [56]: A
```

```
Out[56]: {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
In [57]: sum(A)    # Addition of all elements present in the set A
```

```
Out[57]: 45
```

```
In [58]: max(A)    # Maximum no. present in the Set
```

```
Out[58]: 9
```

```
In [59]: min(A)    # Minimum no. present in the Set
```

```
Out[59]: 1
```

```
In [60]: len(A)
```

Out[60]: 9

```
In [61]: list(enumerate(A))    # Shows index no. and their value Like:- (Index, Value)
```

Out[61]: [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8), (8, 9)]

```
In [62]: D = sorted(A, reverse = True)
D
```

Out[62]: [9, 8, 7, 6, 5, 4, 3, 2, 1]

```
In [63]: sorted(D)
```

Out[63]: [1, 2, 3, 4, 5, 6, 7, 8, 9]

```
In [ ]:
```